

IoT / EMBEDDED SYSTEMS

Daily Technical Report

Project Title	<i>IoT Training Program</i>
Report / Day No.	<i>Day 2</i>
Date	<i>20/7/2026</i>
Prepared By	Malak Alsharqawi
Organization	<i>Cyber Robot</i>

Confidential — Internal Training Documentation

Table of Contents

Table of Contents	2
1. Daily Overview	3
1.1 Summary	3
1.2 Tasks Completed Today	3
Task 1: IR Sensor, LED & Potentiometer — PWM Brightness Control	3
1. Objective	3
2. Procedure	3
Step 1 — Identify the board pinout	3
Step 4 — Wire the LED	3
Step 5 — Combine logic in firmware	3
4. Schematic	4
3. Code / Configuration	4
4. Code Breakdown	5
4.1 Hardcoded configuration & definitions	5
4.2 Microcontroller initialization (setup)	6
4.3 Dynamic sensor acquisition & math	6
4.4 Shared gatekeeper control (if / else)	6
4.5 Telemetry logging & pacing	6
4.6 Why 4095 and 255	6
5. Issues & Troubleshooting	6
6. Results & Outcome	7
2. Key Technical Concepts	7
2.1 ESP32 ADC resolution	7
2.2 PWM frequency and resolution	7
3. Project Documentation & Notes	7

1. Daily Overview

1.1 Summary

This session focused on combining digital sensing and analog control on the ESP32 by wiring an IR obstacle sensor, an LED, and a potentiometer to a single GPIO-based circuit. The IR sensor was used as a presence/proximity gate, while the potentiometer's analog reading was mapped to a PWM duty cycle to adjust the LED's brightness in real time. Along the way, the ESP32's 12-bit ADC behavior and 8-bit PWM resolution were studied, and a boot-time conflict caused by wiring the potentiometer to a strapping pin (GPIO 12) was diagnosed and resolved by moving to a safe ADC1 input pin.

1.2 Tasks Completed Today

- IR Sensor + LED + Potentiometer — PWM Brightness Control
- ESP32 ADC / PWM concept review (12-bit input, 8-bit output)
- Diagnosed and resolved a GPIO strapping-pin upload failure

Task 1: IR Sensor, LED & Potentiometer — PWM Brightness Control

1. Objective

Connect an IR obstacle-avoidance sensor, an LED, and a potentiometer to an ESP32 DEVKIT V1 (DOIT, 36-GPIO variant), then use the potentiometer to adjust LED brightness via PWM only while the IR sensor detects an object in range.

2. Procedure

Step 1 — Identify the board pinout

- Used the ESP32 DEVKIT V1 – DOIT (36-GPIO) pinout diagram to confirm which pins are input-only, which are ADC-capable, and which are unsafe "strapping" pins before wiring anything.

Step 2 — Wire the IR sensor

- Connected the 3-pin IR module (VCC, GND, Vout) to 3.3V, GND, and GPIO 18 (digital input).
- The module's onboard trimpot was used to set the detection distance threshold; the sensor outputs LOW when an object is detected.

Step 3 — Wire the potentiometer

- Connected the potentiometer's outer pins to 3.3V and GND, and its wiper (middle, OUTPUT) pin to GPIO 34, a true input-only ADC1 channel.
- GPIO 34 was chosen instead of an earlier attempt on GPIO 12 specifically to avoid a strapping-pin conflict (see Issues & Troubleshooting).

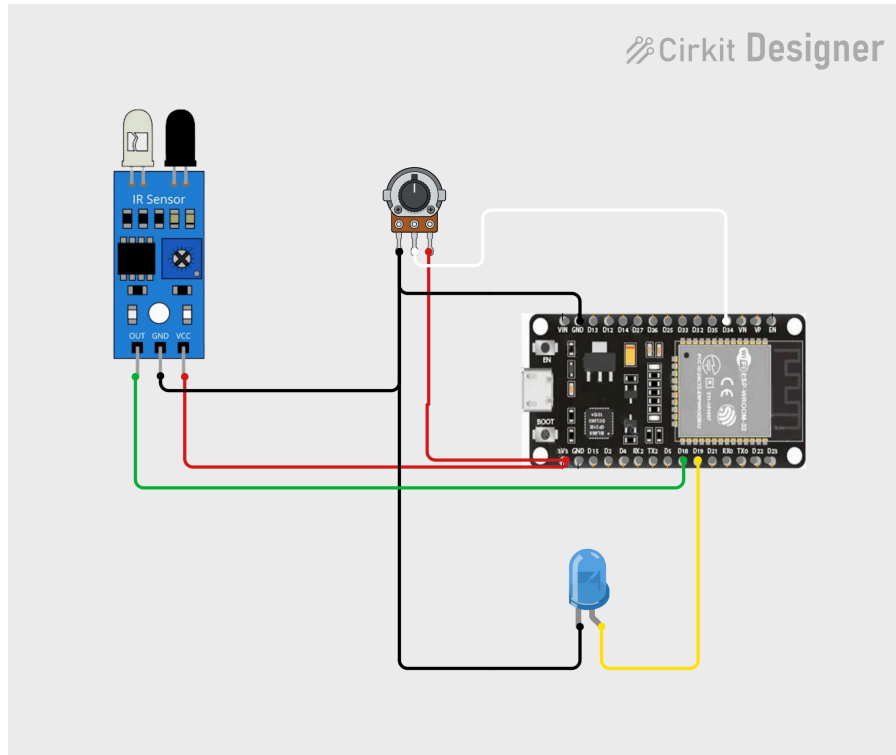
Step 4 — Wire the LED

- Connected the LED (with current-limiting resistor) to GPIO 19, configured as a PWM output via `analogWrite()`.

Step 5 — Combine logic in firmware

- Read the IR sensor digitally and the potentiometer as a 12-bit analog value.
- Mapped the 0–4095 potentiometer range to the 0–255 PWM brightness range using `map()`.
- Drove the LED at the mapped brightness only while the IR sensor reports an object present; otherwise the LED is forced off.

4. Schematic



3. Code / Configuration

main.cpp — full sketch:

```
#include <Arduino.h>

// put function declarations here
#define IR_PIN 18
#define LED_PIN 19
#define POT_PIN 34

int flag = 1;
void setup() {
  // put your setup code here, to run once:
  pinMode(IR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  pinMode(POT_PIN, INPUT);

  Serial.begin(115200);
  Serial.println("Checking IR sensor...");
}
```

```

void loop() {
  int irValue = digitalRead(IR_PIN);
  int potValue = analogRead(POT_PIN);
  int Brightness = map(potValue, 0, 4095, 0, 255);
  float voltage = (potValue / 4095.0) * 3.3; // Calculates actual voltage (0V to
  3.3V)

  if (irValue == LOW) {
    analogWrite(LED_PIN, Brightness);
    if (flag == 0) {
      flag = 1;
      Serial.println("IR sensor detected an object!");
    }
  } else {
    analogWrite(LED_PIN, 0);
    if (flag == 1) {
      flag = 0;
      Serial.println("IR sensor did not detect any object.");
    }
  }

  Serial.print("IR Reading: ");
  Serial.print(irValue == LOW ? "OBJECT" : "CLEAR ");
  Serial.print(" | Pot Value: ");
  Serial.print(potValue);
  Serial.print(" | Voltage: ");
  Serial.print(voltage, 2);
  Serial.println("V");
  delay(100); // Delay for stability
}

```

platformio.ini:

```

[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200

```

monitor_speed was added specifically so the serial monitor output would display cleanly and match the 115200 baud rate set in Serial.begin().

4. Code Breakdown

4.1 Hardcoded configuration & definitions

- #define IR_PIN 18 — the digital input channel for the proximity sensor.
- #define LED_PIN 19 — the physical indicator / dimming LED output.
- #define POT_PIN 34 — maps the potentiometer to a true ADC1 input (ADC1_CH6), avoiding the flashing/boot issues caused by strapping pins.
- int flag = 1; — a binary state marker that ensures each status message prints exactly once per state change, instead of flooding the serial monitor every 100 ms.

4.2 Microcontroller initialization (setup)

- `pinMode(...)` — sets explicit electrical directions (INPUT/OUTPUT) for all three pins.
- `Serial.begin(115200);` — opens the data link between the ESP32 and the computer at 115,200 baud.

4.3 Dynamic sensor acquisition & math

- `analogRead(POT_PIN)` — reads the knob's physical position as a value from 0 (0V) to 4095 (3.3V).
- `map(potValue, 0, 4095, 0, 255)` — rescales the 12-bit analog reading down to the 8-bit range `analogWrite()` expects for PWM.
- `float voltage = (potValue / 4095.0) * 3.3;` — converts the raw ADC step back into a human-readable voltage.

4.4 Shared gatekeeper control (if/ else)

- `analogWrite(LED_PIN, Brightness)` — when the IR sensor reads LOW (object present), the LED is driven at the duty cycle set by the potentiometer.
- The `flag == 0` guard — prints "object detected" only once, at the moment the object is first sensed.
- `analogWrite(LED_PIN, 0)` — when the path is clear, the LED is forced fully off regardless of the potentiometer position.
- The `flag == 1` guard — prints a single exit message once the object leaves, and resets the flag.

4.5 Telemetry logging & pacing

- The ternary `irValue == LOW ? "OBJECT" : "CLEAR"` converts the raw 0/1 reading into a readable label for the serial log.
- `Serial.print(voltage, 2)` truncates the float to 2 decimal places (e.g. 1.65V).
- `delay(100)` paces the log output to roughly 10 updates per second, keeping the serial monitor readable.

4.6 Why 4095 and 255

- 4095 is the ESP32 ADC's maximum step: its 12-bit ADC produces $2^{12} = 4096$ discrete steps (0–4095), representing 0V to 3.3V.
- 255 is the PWM output ceiling: `analogWrite()` uses an 8-bit duty-cycle resolution, giving $2^8 = 256$ steps (0–255), where 255 is 100% duty cycle.
- `map(potValue, 0, 4095, 0, 255)` is the translator that rescales the 0–4095 knob position onto the 0–255 LED power scale.

5. Issues & Troubleshooting

Symptom:

Failed to communicate with the flash chip, read/write operations will fail

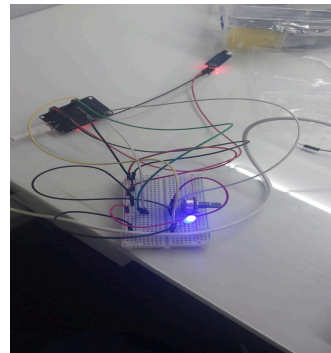
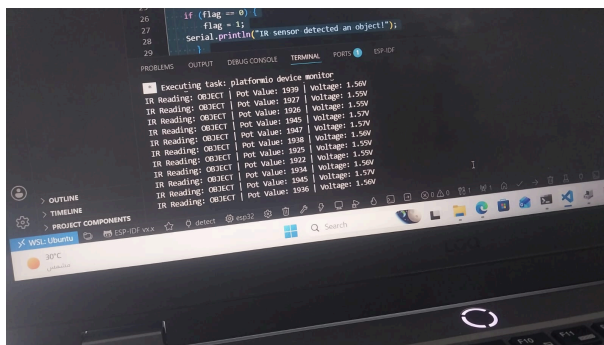
Root cause — GPIO 12 is a strapping pin: the ESP32 samples the electrical state of certain "strapping" pins the instant it powers on or enters flashing mode. GPIO 12 specifically sets the internal flash chip's operating voltage — LOW at boot selects 3.3V, HIGH forces 1.8V. The potentiometer that was originally wired to GPIO 12 could leave that pin HIGH depending on knob position, so a reboot or upload while the

dial sat in the wrong range forced the flash chip into 1.8V mode. Since the flash actually needs 3.3V, it was starved of power and the upload tool lost communication with it.

Resolution: moved the potentiometer's wiper from GPIO 12 to GPIO 34 (an input-only ADC1 pin with no boot-strapping role), which immediately restored reliable uploading and flashing.

6. Results & Outcome

The circuit correctly gated LED brightness on IR sensor state: with an object in range, the LED brightness tracked the potentiometer smoothly from 0 to 255 (via the 0–4095 → 0–255 map); with the path clear, the LED switched off regardless of the potentiometer position. Serial output at 115200 baud confirmed IR state ("OBJECT"/"CLEAR"), the raw potentiometer value, and the computed voltage (2 decimal places), updating roughly every 100 ms. Moving the potentiometer off GPIO 12 to GPIO 34 eliminated the flash-communication upload failure.



2. Key Technical Concepts

2.1 ESP32 ADC resolution

Unlike the classic Arduino Uno's 10-bit ADC (0–1023), the ESP32 uses a 12-bit ADC, so `analogRead()` returns a range from 0 to 4095.

2.2 PWM frequency and resolution

- **PWM Frequency** — the number of times a PWM signal completes one full ON/OFF cycle per second.
- **PWM Resolution** — the granularity of duty-cycle steps available; a higher resolution allows finer control over brightness or speed.

3. Project Documentation & Notes

- **Development Notes & Key Learnings:**
https://app.notion.com/p/Day-1-3a254ef921fa80769a0dc1db6c03625e?source=copy_link (shared Day 1/2 Notion notebook)
- **ESP32 Pinout Reference (GPIOs):** <https://randomnerdtutorials.com/esp32-pinout-reference-gpios/> — verified live; used to select safe ADC/strapping-free pins
- **The Potentiometer: Pinout, Wiring, and How It Works:**
<https://www.build-electronic-circuits.com/potentiometer/> — verified live; background on 3-pin wiring

- **Arduino LED Dimmer (Potentiometer + PWM):**
<https://leecuriosity.com/arduino-led-dimmer-potentiometer-pwm/> — *verified live; reference for the map()-based dimming approach*

End of Report